

## Expt 9: Banker's Algorithm:

[Back To Practice](#)

Completed

### Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

#### Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.

Select Language: C (GCC 9.2.0)

```
1 #include<stdio.h>
2 int main(){
3     int n,r;
4     printf("Enter number of processes: \n");
5     scanf("%d",&n);
6     printf("Enter number of resource types: \n");
7     scanf("%d",&r);
8     int alloc[n][r],max[n][r],need[n][r],avail[r];
9     printf("Enter Allocation Matrix:\n");
10    for(int i = 0; i < n; i++)
11        for(int j = 0; j < r; j++)
12            scanf("%d",&alloc[i][j]);
13    printf("Enter Max Matrix:\n");
14    for(int i = 0; i < n; i++)
15        for(int j = 0; j < r; j++)
16            scanf("%d", &max[i][j]);
17    printf("Enter Available Resources: \n");
```

Test Cases 2

Run Sample Cases Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

### Output Format

The output displays whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

#### Example 1

##### Sample Input 1

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
```

Select Language: C (GCC 9.2.0)

```
17    printf("Enter Available Resources: \n");
18    for(int j = 0; j < r; j++)
19        scanf("%d", &avail[j]);
20    for(int i = 0; i < n; i++)
21        for (int j = 0; j < r; j++)
22            need[i][j] = max[i][j] - alloc[i][j];
23    int finish[n], safe_seq[n];
24    for(int i = 0; i < n; i++)
25        finish[i] = 0;
26    int work[r];
27    for(int j = 0; j < r; j++)
28        work[j] = avail[j];
29    int count = 0;
30    while(count < n) {
31        int found = 0;
32        for(int i = 0; i < n; i++) {
33            if(finish[i])
```

Test Cases 2

Run Sample Cases Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

[Back To Practice](#)

Q1

Save

Select Language: C (GCC 9.2.0)

```

33         if(!finish[i])
34             continue;
35         int can_allocate = 1;
36         for(int j = 0; j < r; j++) {
37             if(need[i][j] > work[j]) {
38                 can_allocate = 0;
39                 break;
40             }
41         }
42         if(can_allocate) {
43             for(int j = 0; j < r; j++)
44                 work[j] += alloc[i][j];
45             safe_seq[count++] = i;
46             finish[i] = 1;
47             found = 1;
48         }
49     }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

3 0 3

3 2 2

2 1 1

4 3 3

0 0 0

Sample Input 1

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

[Back To Practice](#)

Q1

Save

Select Language: C (GCC 9.2.0)

```

50         if(!found)
51             break;
52     }
53     if(count == n) {
54         printf("System is in a SAFE STATE.\n");
55         printf("Safe Sequence: ");
56         for(int i = 0; i < n; i++){
57             printf("%d", safe_seq[i]);
58             if(i < n - 1)
59                 printf(" ");
60         }
61         printf("\n");
62     }else {
63         printf("System is NOT in a safe state (Deadlock may occur).\n");
64     }
65     return 0;
66 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

3

2

10

0 1

1 1

2 1

1 2

3 3

1 1

Sample Input 2

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

[Back To Practice](#)

Q1

Save

Success

Custom Test

Input :

3

3

0 1 0

2 0 0

3 0 3

3 2 2

2 1 1

4 3 3

0 0 0

Expected Output :

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

Output :

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

1 1

2 1

1 2

3 3

1 1

Sample Input 2

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is in a SAFE STATE.

Safe Sequence: P0 P1 P2